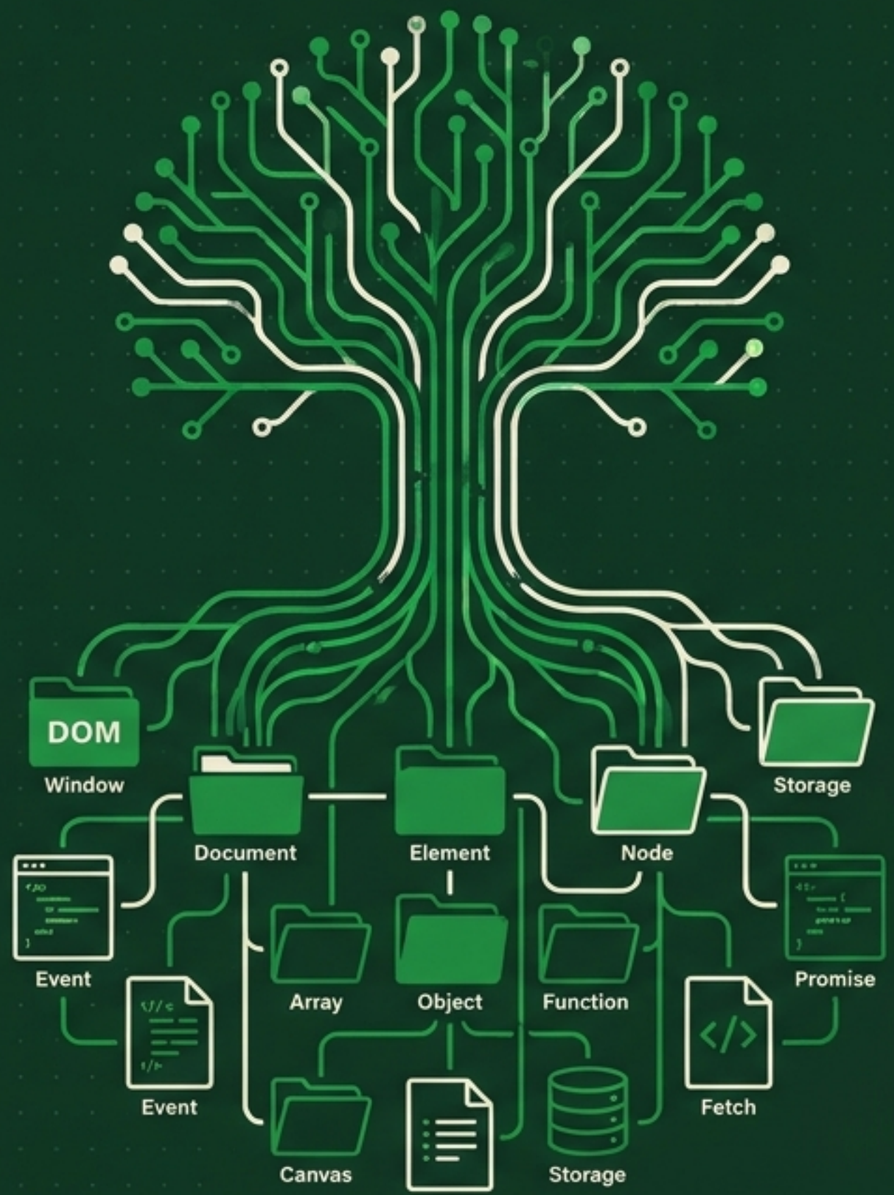


JavaScript: La Guía de Campo del Desarrollador

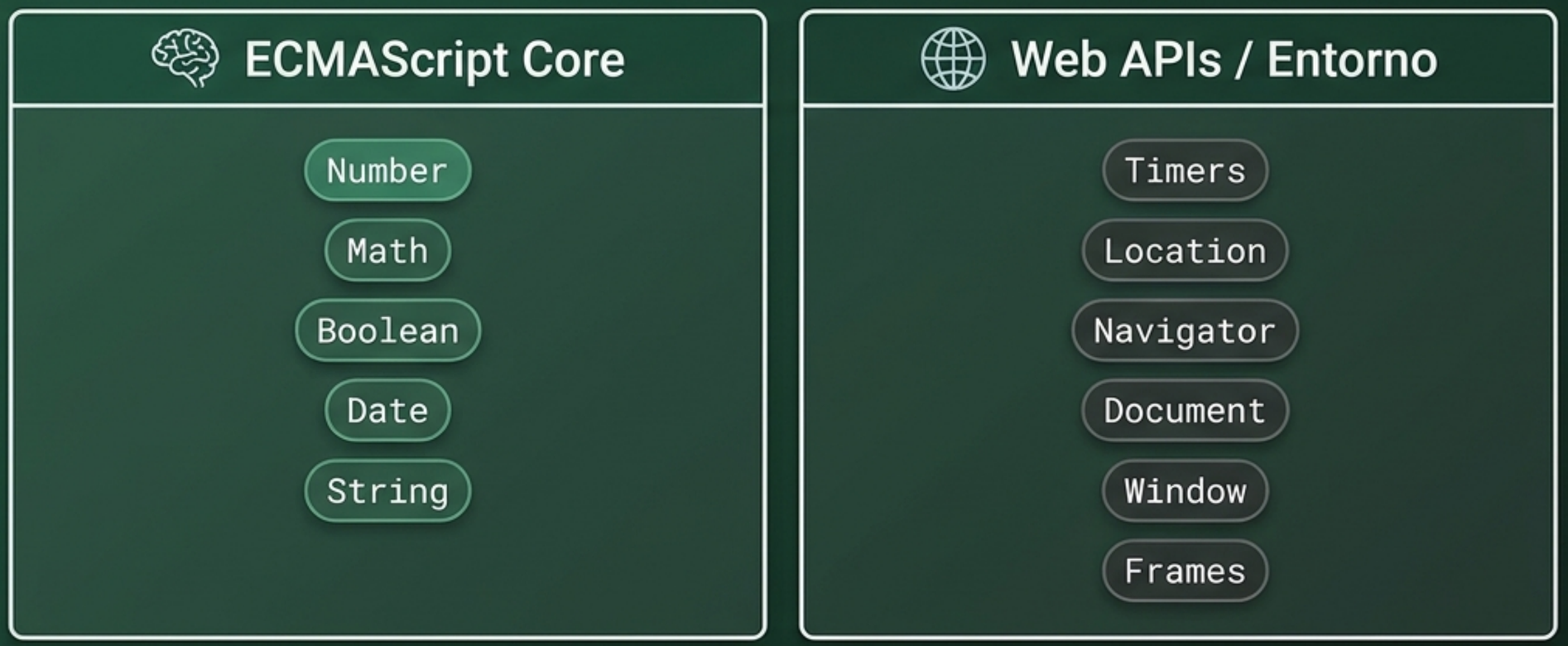
Dominando los Objetos Predefinidos y Web APIs

Una clase magistral de estructuración, manipulación y renderizado de datos en el ecosistema web.

Gemini Notebook

El Modelo Mental de JavaScript

Para dominar JS, debes separar mentalmente lo que pertenece al lenguaje puro de lo que proporciona el navegador.



💡 **[PRO TIP]** Los objetos Core funcionan en cualquier entorno (como Node.js), mientras que las Web APIs son los 'puentes' exclusivos hacia el navegador.

Gemini Notebook

1. Objeto: Number

Objeto primitivo envolvente para representar y manipular valores numéricos.

💡 En el mundo real...

Usado para forzar la conversión de tipos desde inputs de texto (`Number("123")`) y validar límites matemáticos.

Método	¿Para qué sirve?	Ejemplo Real
`Number.MAX_SAFE_INTEGER`	Límite seguro (9007199254740991)	`const max = Number.MAX_SAFE_INTEGER;`
`Number.parseFloat()`	Extrae decimales de un string	`Number.parseFloat("12.3") // 12.3`
`Number.isNaN()`	Verifica si no es un número	`Number.isNaN(NaN) // true`

⚠️ [WARNING] Evita usar `new Number("123")`. Crea un objeto (typeof === 'object''), no un primitivo. Para números mayores al límite seguro, usa `BigInt`.

Gemini Notebook

2. Objeto: Math

Un objeto estático incorporado que agrupa constantes y funciones matemáticas. No es una función ni requiere instanciación (no funciona con BigInt).

💡 En el mundo real...

Esencial para utilidades algorítmicas, trigonometría en Canvas/juegos, y generación de números pseudo-aleatorios (ej. lanzar dados, tokens).

Método	¿Para qué sirve?	Ejemplo Real
`Math.random()`	Genera valor entre 0 y 1	`Math.random()`
`Math.floor()`	Redondea hacia abajo	`Math.floor(4.9) // 4`
`Math.max()` / `Math.min()`	Devuelve el mayor/menor	`Math.max(10, 20) // 20`

💡 **[PRO TIP]** Math puede ser extendido directamente (sin prototype). Ej: `Math.gcd = function(a, b) { ... }` para calcular el máximo común divisor.

Gemini Notebook

3. Objeto: Boolean

Objeto contenedor para un valor lógico (`true` o `false`). Esencial para el control de flujo (condicionales) y la coerción de tipos.

⚠️ [WARNING] Anti-patrón

```
var x = new Boolean(false);
```

¡Cualquier objeto se evalúa como true! `if (x)` se ejecutará aunque su valor interno sea falso.

✅ Práctica Real

```
var x = Boolean(expression);

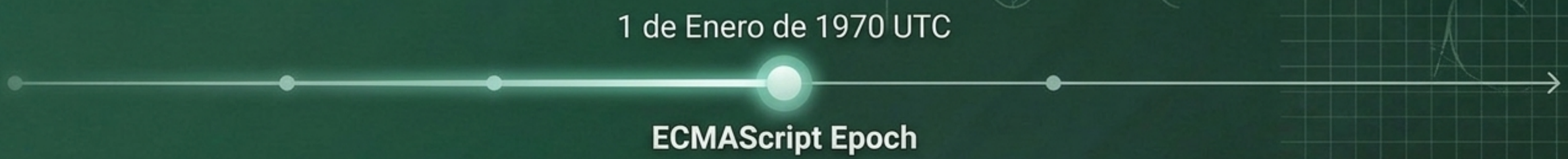
var x = !!expression;
```

Usa la función de conversión directa o el doble NOT lógico para primitivas seguras.

Métodos Clave:	`Boolean.prototype.toString()` | Devuelve 'true' o 'false'. `Boolean.prototype.valueOf()` | Devuelve el valor primitivo.

Gemini Notebook

4. Objeto: Date



Representa un momento fijo en milisegundos desde este punto.

💡 En el mundo real...

Se usa para marcas temporales (timestamps), calcular tiempo transcurrido y formatear fechas para la interfaz local.

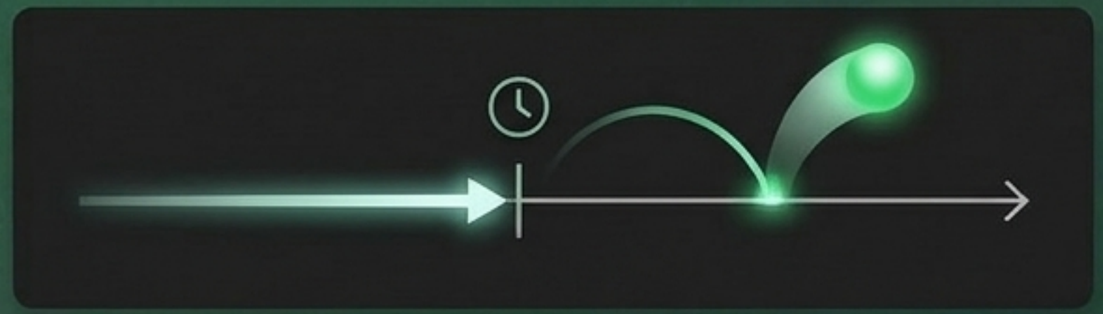
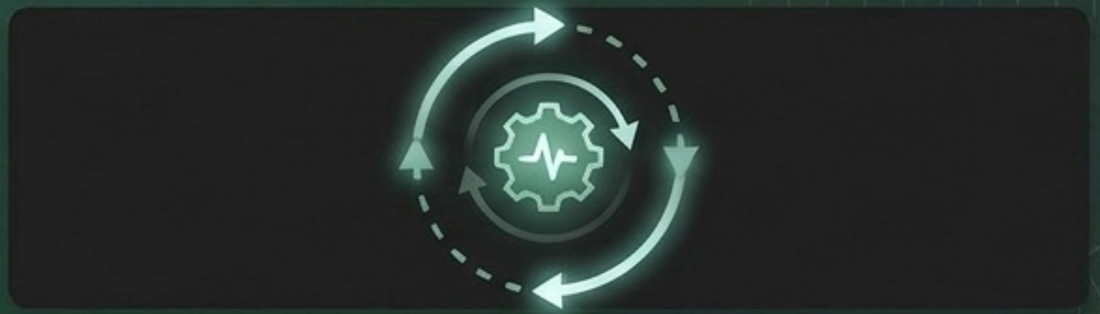
Método	¿Para qué sirve?	Ejemplo Real
`Date.now()`	Milisegundos desde Epoch	`Date.now()`
`getFullYear()`	Año de 4 dígitos (Local)	`date.getFullYear()`
`toISOString()`	Formato extendido estándar	`date.toISOString()`

⚠️ **[WARNING]** Los métodos base operan en la Zona Horaria Local. Usa `getUTCFullYear()` para hora universal. Evita `getYear()` (hereda un bug de 2 dígitos de 1900); usa siempre `getFullYear()`.

Gemini Notebook

5. Intervalos de tiempo

Herramientas de temporización asíncrona gestionadas por el Event Loop del navegador.
Útiles para diferir ejecuciones o animaciones simples.

setTimeout(fn, ms)	setInterval(fn, ms)
	
Ejecuta una sola vez tras "ms". Ej: Notificaciones temporales.	Ejecuta repetidamente cada "ms". Ej: Reloj en pantalla, polling.
Limpieza: `clearTimeout(id)` / `clearInterval(id)` cancelan la ejecución.	

⚠ [WARNING]	El tiempo en ms es un mínimo, no exacto. **Antipatrón**: Olvidar limpiar intervalos causa fugas de memoria (Memory Leaks).

Gemini Notebook

6. Objeto: String

Representa y manipula una secuencia de caracteres (unidades de código UTF-16).
Usado para limpieza de inputs, análisis de URLs, y UI dinámica.

```
1  // Template Literals en el mundo real
2  const user = 'Desarrollador';
3  const msg = `Bienvenido, ${user.trim()}.`;
```

Método	¿Para qué sirve?	Ejemplo Real
`length`	Longitud de cadena	`str.length`
`slice(start, end)`	Extrae sección	`str.slice(0, 5)`
`split(sep)`	Divide en arreglo	`str.split(',')`
`trim()`	Limpia espacios	`str.trim()`

💡 **[PRO TIP]** Cuidado con `eval()`. `eval('2+2')` devuelve 4, pero `eval(new String('2+2'))` devuelve el objeto String '2+2'. Usa primitivas.

Gemini Notebook

7. Objeto: Location

Proporciona información sobre la URL actual del documento. En el mundo real: redirecciones dinámicas, recargas, o lectura de parámetros (?id=123).



Método	¿Para qué sirve?	Ejemplo Real
`location.href`	URL completa	`console.log(location.href)`
`location.assign(url)`	Navega a nueva URL (guarda historial)	`location.assign('/home')`
`location.replace(url)`	Reemplaza URL (sin historial)	`location.replace('/login')`
`location.reload()`	Refresca la página	`location.reload()`

Gemini Notebook

8. Objeto: Navigator

Representa el estado y la identidad del User Agent (el navegador) y el dispositivo físico del usuario.

En el mundo real... Detección de idioma preferido, verificación de conexión a internet, o acceso a hardware (cámara, GPS).



`navigator.userAgent`

Identidad del navegador

`Mozilla/5.0...`



`navigator.language`

Idioma del sistema

`es-ES`



`navigator.geolocation`

API de ubicación física

`getCurrentPosition()`



`navigator.onLine`

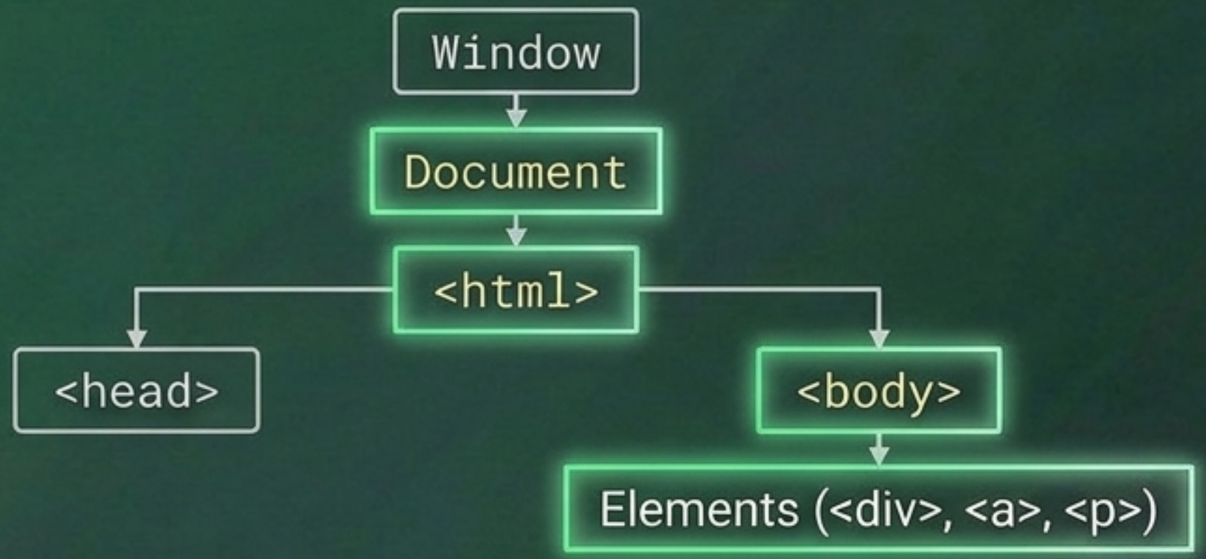
Estado de red

`true/false`

Gemini Notebook

9. Objeto: Document (DOM)

Hace referencia al documento mismo cargado en la ventana (Document Object Model). Convierte HTML estático en un árbol de nodos manipulable para interactuar con la UI.



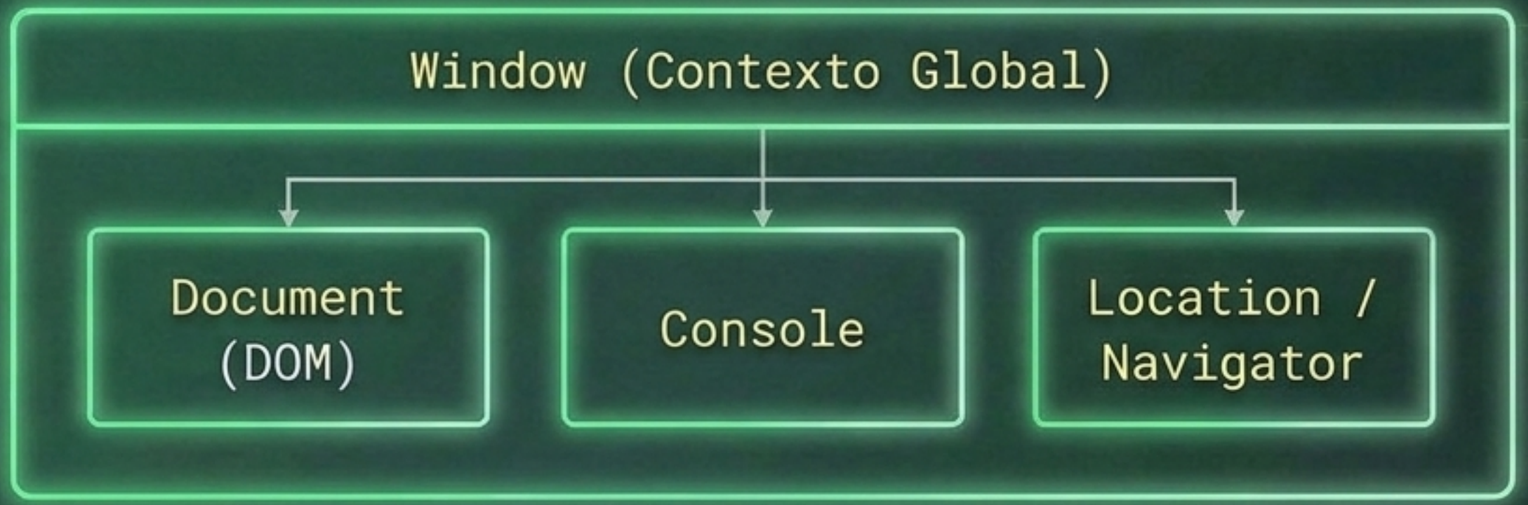
Método	¿Para qué sirve?	Ejemplo Real
`getElementById()`	Selecciona nodo por ID	`document.getElementById('btn')`
`querySelector()`	Selecciona por regla CSS	
`createElement()`	Crea un nuevo nodo HTML	

💡 **[PRO TIP]** En el DOM rudimentario existían **colecciones** históricas estáticas; hoy dominan los métodos selectores dinámicos.

Gemini Notebook

10. Objeto: Window

Teoría: Es el objeto raíz. Representa la pestaña/ventana del navegador que contiene el documento DOM. Actúa como el contexto global de JavaScript.



Método	Descripción
`window.innerWidth/Height`	Tamaño del viewport visible.
`window.addEventListener('resize')`	Evento lanzado al modificar tamaño.
`window.setTimeout()`	Reside en Window globalmente.

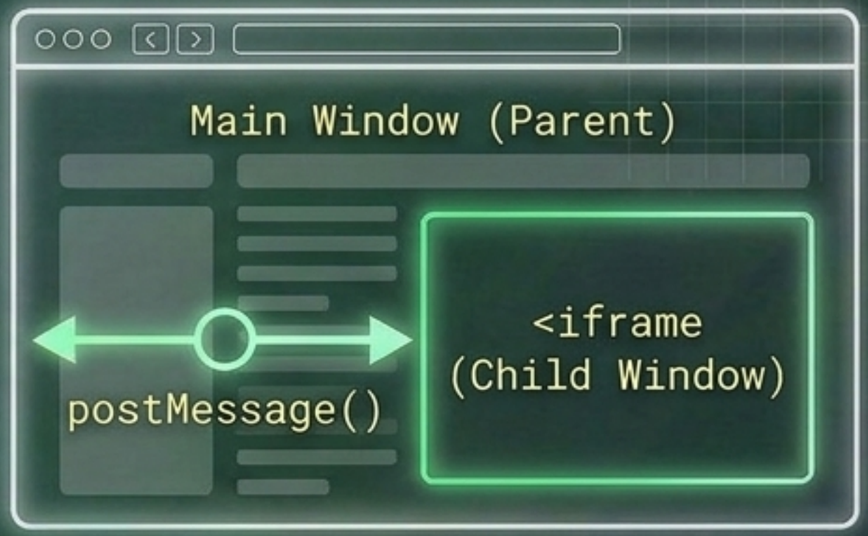
💡 **[WARNING]** Al declarar 'var x = 1' en el entorno raíz, estás mutando directamente el objeto 'window.x'.

Gemini Notebook

11. Objetos: Frames (y window.frames)

Teoría: Una colección o pseudo-arreglo que devuelve todas las sub-ventanas (**<iframe>**) integradas. Cada frame tiene su propio objeto Window.

💡 **[En el mundo real...]** Comunicación segura (cross-frame messaging) y carga de widgets de terceros (ej. pasarelas de pago).

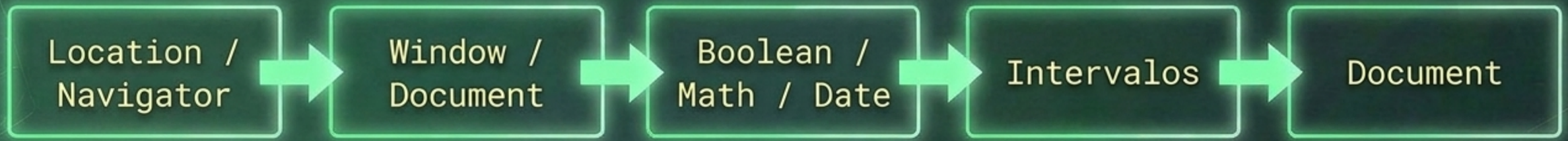


Método	Descripción	Ejemplo Real
`window.frames`	Array-like object de iframes	`window.frames[0]`
`window.length`	Cant. de frames en la página	`if (window.length > 0)`
`postMessage()`	Comunicación segura	Transmite datos sin romper CORS.

Gemini Notebook

Síntesis: Anatomía de una Aplicación Web

¿Cómo interactúan estas 11 herramientas juntas en código de producción?



Detectamos de dónde viene el usuario y su idioma.

El usuario introduce un String o Number en un formulario del DOM.

Validamos (Boolean), calculamos (Math), y estampamos hora (Date).

Mostramos un loader asíncrono con setInterval.

Renderizamos el resultado de vuelta al árbol DOM.

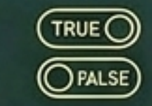
Gemini Notebook

La Hoja de Referencia del Desarrollador (Cheat Sheet)

Number
Valores & Límites

Math

Algoritmos

Boolean


Lógica & Coerción

Date

Tiempo Universal

Timers

Asincronía

String

Texto & Plantillas

Location

Rutas

Navigator

Dispositivo

Document

Árbol UI

Window

Contexto Global

Frames

Sub-entornos

Guarda esta ficha. Conocer tus herramientas define qué tan lejos puedes construir.
Fin de la exposición.

Gemini Notebook